

UNIVERSITAS NEGERI YOGYAKARTA
FAKULTAS TEKNIK
PROGRAM STUDI TEKNIK INDUSTRI

LAPORAN PRAKTIKUM
APLIKASI WEB DAN MOBILE

Semester Genap 2026/2027

MODUL 8- Asynchronous JavaScript

Nama Mahasiswa : Aisyah Ayu Salsabila
NIM : 23051430001
Kelas : A2_Teknik Industri
Tanggal Praktikum : 16 April 2026
Dosen Pengampu : Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT.

Dikumpulkan paling lambat 7 hari setelah pelaksanaan praktikum

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 8 (Pertemuan 8)
Topik Utama	Asynchronous JavaScript
Sub-Topik	Konsep Asynchronous, Fetch API, Promise, Async/Await, dan integrasi REST API eksternal.
Durasi Praktikum	340 Menit
Tanggal Praktikum	16 April 2026
Nama Mahasiswa	Aisyah Ayu Salsabila
NIM	23051430001
Kelas	A2_Teknik Industri

B. TUJUAN PEMBELAJARAN

1	Memahami perbedaan kode sinkronus dan asinkronus.
2	Mampu menggunakan `fetch()` untuk mengambil data dari server eksternal (API).
3	Memahami format data JSON (JavaScript Object Notation).
4	Menerapkan penanganan error saat request API gagal.

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	Ekstensi Live Server		Perubahan web realtime
4	Laptop	Windows 11	Prasarana Praktikum
5	Modul Aplikasi Web dan Mobile	2025/2026	Manual Praktikum
6	Console browser dan ekstensi Quokka.js		JavaScript alat pemroses data

7.	Internet		Mengakses API publik
8.	Postman		Testing API

D. DASAR TEORI

1. Latar Belakang Pemrograman Web dalam Konteks Teknik Industri

Teknik Industri berfokus pada efisiensi sistem terintegrasi yang melibatkan aliran informasi secara real-time. Dalam era digital, terutama dengan berkembangnya konsep Industrial Internet of Things (IIoT), data tidak lagi tersimpan secara lokal, melainkan tersebar di berbagai server dan sistem berbasis cloud.

Sebagai contoh, data stok gudang dapat berada di server pusat (misalnya Jakarta), sementara operator atau manajer produksi mengaksesnya dari lokasi berbeda (misalnya Surabaya). Hal ini menuntut sistem yang mampu mengambil data secara cepat tanpa mengganggu proses lain.

Dalam pemrograman web, kebutuhan ini diwujudkan melalui konsep asynchronous programming dan penggunaan Application Programming Interface (API). Keduanya memungkinkan sistem untuk tetap berjalan responsif meskipun sedang mengambil atau mengirim data ke server eksternal.

2. Asynchronous vs Synchronous sebagai Representasi Aliran Proses Industri

Dalam sistem industri, terdapat perbedaan antara proses yang berjalan secara berurutan dan proses yang dapat berjalan paralel.

- Synchronous: Proses berjalan berurutan (seperti antrian produksi). Jika menunggu server, sistem bisa “freeze”.
- Asynchronous: Proses tidak perlu menunggu (seperti produksi paralel).

Program tetap berjalan, dan respon diproses saat data sudah diterima.

Dengan pendekatan asynchronous, sistem menjadi lebih efisien dan responsif, seperti halnya sistem produksi yang dirancang untuk meminimalkan waktu tunggu (waiting time).

3. API sebagai Jembatan Pertukaran Data Antar Sistem

API berfungsi sebagai jembatan komunikasi antara frontend dan server.

- Analogi: Jalur distribusi antar bagian dalam industri
- Data biasanya dalam format JSON
- Digunakan untuk mengambil, mengirim, dan mengelola data

4. Fetch API

Dalam pemrograman web modern, Fetch API digunakan untuk melakukan komunikasi dengan server melalui HTTP request.

Fetch API (Pemrograman)

Analogi Industri: Operator yang meminta data ke pusat kontrol tanpa menghentikan pekerjaannya. Berbasis asynchronous (menggunakan Promise)

Mendukung berbagai metode HTTP:

GET → Mengambil data (misalnya melihat stok)

POST → Mengirim data baru (input produksi)

PUT → Memperbarui data

DELETE → Menghapus data

Keunggulan Fetch API adalah lebih modern dibanding XMLHttpRequest, mendukung async/await (lebih mudah dibaca) dan tidak memblokir eksekusi program.

D.1 Konsep Utama

Asynchronous tetap berjalan dengan baik suatu aplikasi tanpa menunggu, dengan menggunakan aktivasi, tanpa antrian, Solusi bottleneck, JSON untuk komunikasi dengan server, membangun struktur dan melakukan promise

D.2 Keterkaitan dengan Teknik Industri

Asynchronous programming dalam web mirip dengan sistem produksi paralel di industri: proses tetap berjalan tanpa harus menunggu satu tugas selesai. **Fetch API** berperan seperti operator yang mengambil data dari pusat kontrol, dimana metode GET untuk melihat stok dan POST untuk mengirim laporan produksi.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Mengenal JSONPlaceholder

Kita akan menggunakan layanan gratis `'https://jsonplaceholder.typicode.com/users'` sebagai simulasi database "Karyawan" atau "Agen Supplier".

Format akan dianali, setiap mengirim data, notifikasi akan dikenali, untuk masuk pada entity.

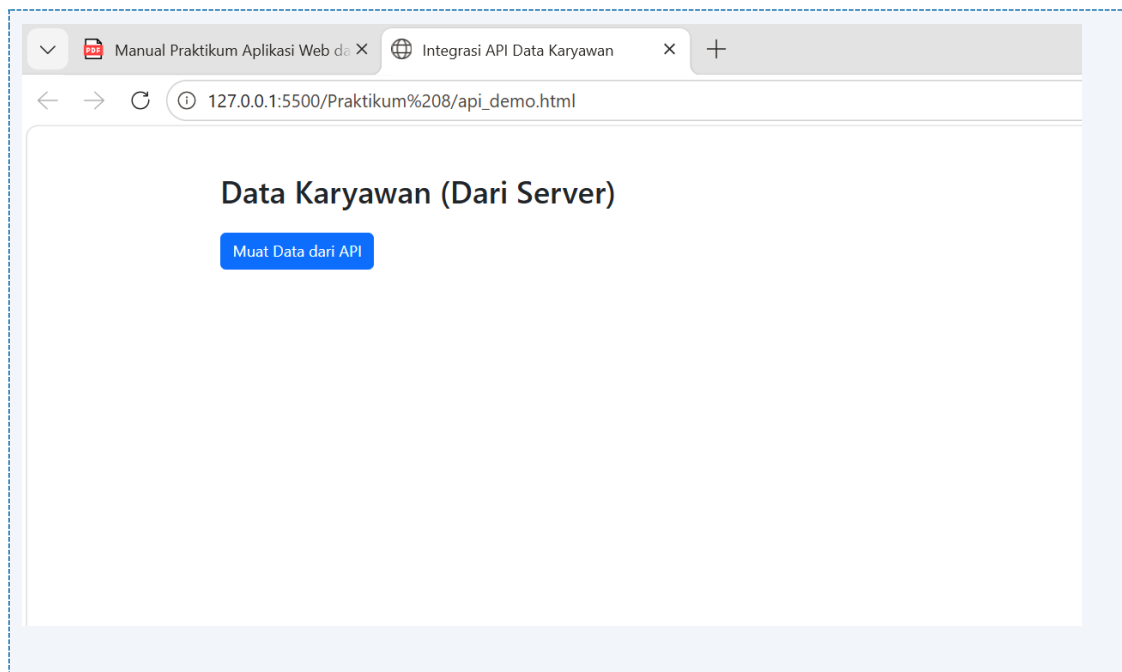
E.2 Langkah 2 – Membuat HTML untuk Menampilkan Data

Membuat `api_demo.html`

```
<!DOCTYPE html>
<html lang="id">
<head>
<meta charset="UTF-8">
<title>Integrasi API Data Karyawan</title>
<link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body class="container mt-5">
<h2 class="mb-4">Data Karyawan (Dari Server)</h2>
<div class="mb-3">
<button id="btnLoad" class="btn btn-primary">Muat Data dari API</button>
```

```
<div id="loading" class="spinner-border text-primary d-none"
role="status"></div>
</div>
<div class="row" id="containerKaryawan">
<!-- Card karyawan akan muncul di sini -->
</div>
<script src="api_script.js"></script>
</body>
</html>
```

Screenshot Hasil:



Gambar E.1 – API html

E.3 Langkah 3 – Fetch Data dengan .then() (Promise)

Membuat `api\_script.js`, penyederhanaan penulisan

```
const btnLoad = document.getElementById('btnLoad');
const container = document.getElementById('containerKaryawan');
const loading = document.getElementById('loading');

// Endpoint API (Simulasi Database)
const API_URL = 'https://jsonplaceholder.typicode.com/users';

btnLoad.addEventListener('click', function() {
  // Tampilkan loading
  loading.classList.remove('d-none');
  container.innerHTML = ''; // Bersihkan konten lama
```

```
// Fetch Data
fetch(API_URL)
  .then(function(response) {
    // Cek jika response sukses (kode 200-299)
    if (!response.ok) {
      throw new Error('Gagal mengambil data'); // Perbaikan:
// pindahkan ke dalam if
    }
    // Parsing data JSON
    return response.json();
  })
  .then(function(dataKaryawan) {
    // Data berhasil didapat
    console.log(dataKaryawan); // Cek di console browser
    renderData(dataKaryawan);
  })
  .catch(function(error) {
    // Jika ada error (misal putus internet)
    container.innerHTML = `<div class="alert alert-danger">Error:
${error.message}</div>`;
  })
  .finally(function() {
    // Sembunyikan loading (baik sukses maupun gagal)
    loading.classList.add('d-none');
  });
});

function renderData(data) {
  if (!data || data.length === 0) {
    container.innerHTML = `<div class="alert alert-warning">Tidak ada
data karyawan</div>`;
    return;
  }

  data.forEach(function(karyawan) {
    // Buat elemen card Bootstrap
    const col = document.createElement('div');
    col.className = 'col-md-4 mb-3';

    // Gunakan optional chaining untuk menghindari error jika properti
    // tidak ada
    const companyName = karyawan.company?.name || 'Tidak tersedia';
    const cityName = karyawan.address?.city || 'Tidak tersedia';

    col.innerHTML = `
      <div class="card h-100 shadow-sm">
```

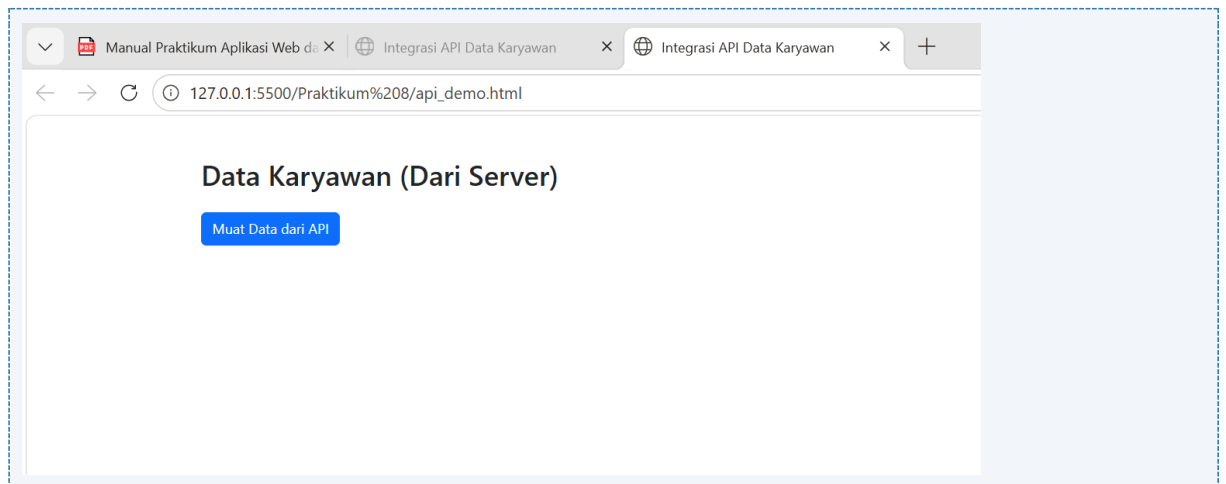
```

        <div class="card-body">
            <h5 class="card-title">${karyawan.name}</h5>
            <p class="card-text text-muted">Email:
${karyawan.email}</p>
            <p class="card-text">Perusahaan: ${companyName}</p>
            <p class="card-text"><small>Kota:
${cityName}</small></p>
            <button class="btn btn-sm btn-outline-primary"
onclick="cariKaryawan(${karyawan.id})">Detail Profil</button>
        </div>
    </div>
    `;

    container.appendChild(col);
});
}

```

Screenshot Hasil:



Gambar E.2 – API html dari js

E.4 Langkah 4 – Fetch dengan Async/Await (Modern Style)

Async/Await membuat kode asinkron terlihat seperti sinkron (lebih mudah dibaca). Mari kita buat fungsi baru untuk mencari karyawan berdasarkan ID. Penambahan pada api_script.js

```

// Fungsi Async untuk mencari data spesifik
async function cariKaryawan(id) {
    try {
        console.log(`Mencari data ID: ${id}...`);

        const response = await
fetch(`https://jsonplaceholder.typicode.com/users/${id}`);

```

```

    if (!response.ok) {
        throw new Error('Data tidak ditemukan');
    }

    const data = await response.json();
    console.log("Ditemukan:", data);

    // Tampilkan detail dalam modal atau alert yang lebih informatif
    alert(`Ditemukan:\nNama: ${data.name}\nEmail:
    ${data.email}\nPerusahaan: ${data.company?.name || '-'}\nKota:
    ${data.address?.city || '-'} `);

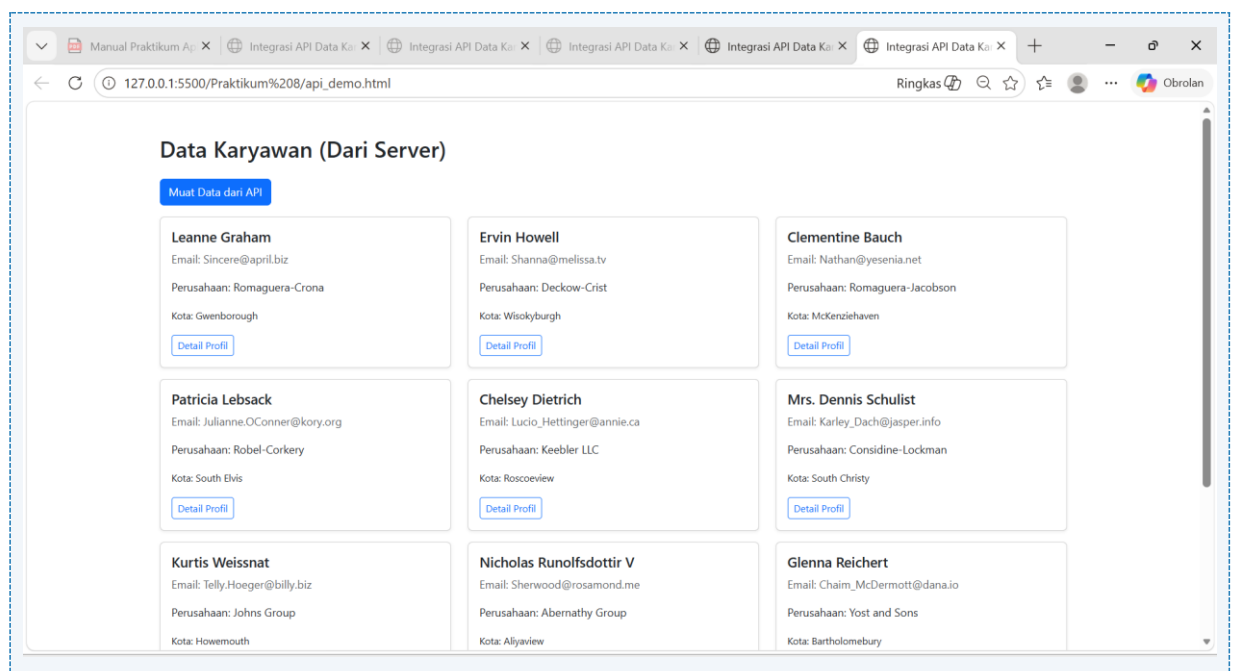
    return data; // Mengembalikan data agar bisa digunakan oleh fungsi
    lain

} catch (error) {
    console.error(error);
    alert(`Error: ${error.message}`);
}
}

// Contoh pemanggilan: cariKaryawan(2);
// Anda bisa memanggil fungsi ini lewat console browser untuk tes.

```

Screenshot Hasil:



Gambar E.3 – API html dengan fetch

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Posting Data

Pertanyaan / Instruksi Latihan:

Latihan 1 (Posting Data): Buat tombol "Tambah Karyawan Baru". Gunakan method `fetch` dengan opsi `{ method: 'POST', body: ... }` untuk mengirim data dummy ke API. Cek response di console.

Jawaban / Kode Solusi:

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Integrasi API Data Karyawan</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.
css"
rel="stylesheet">
</head>
<body class="container mt-5">
  <h2 class="mb-4">Data Karyawan (Dari Server)</h2>
  <div class="mb-3">
    <button id="btnLoad" class="btn btn-primary">Muat Data dari
API</button>
    <div id="loading" class="spinner-border text-primary d-none"
role="status"></div>
  </div>
  <div class="row" id="containerKaryawan">
    <!-- Card karyawan akan muncul di sini -->
  </div>

  <script src="Latihan1.js"></script>
</body>
</html>
```

Jawaban untuk JS:

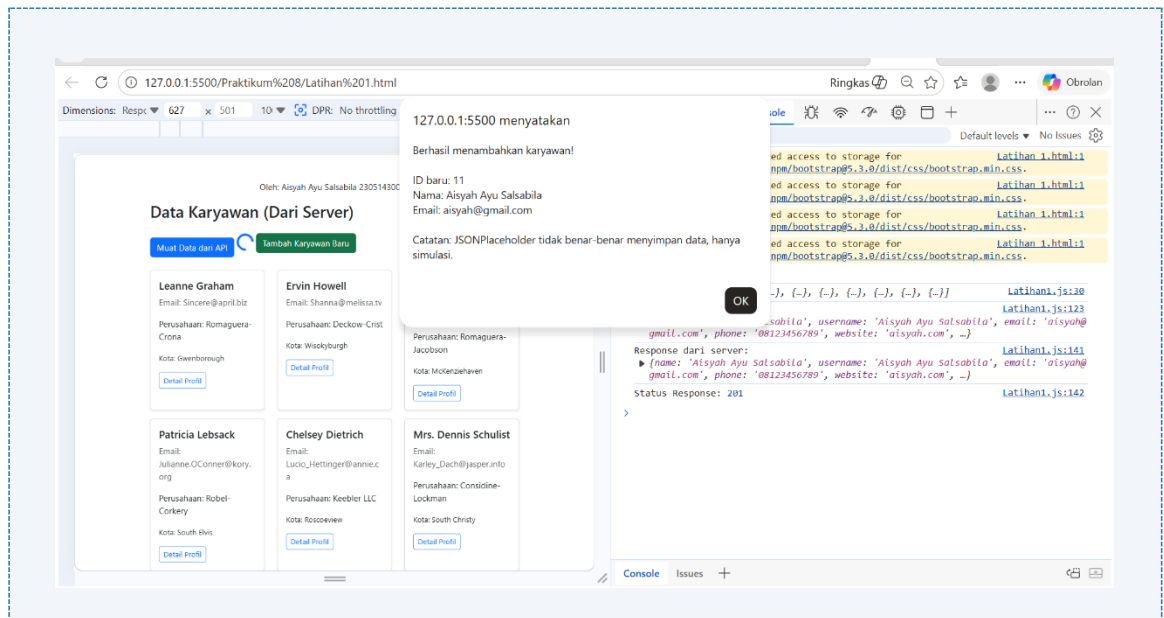
```
// Buat tombol Tambah
const btnTambah = document.createElement('button');
btnTambah.textContent = 'Tambah Karyawan Baru';
btnTambah.className = 'btn btn-success mb-3 me-2';
btnTambah.id = 'btnTambah';
const parentDiv = btnLoad.parentElement;parentDiv.appendChild(btnTambah);
// Event listener untuk tombol Tambah
btnTambah.addEventListener('click', function() {
  tambahKaryawanBaru();
});
```

```
});  
async function tambahKaryawanBaru() {  
  const karyawanBaru = {  
    name: "Aisyah Ayu Salsabila",  
    username: "Aisyah Ayu Salsabila",  
    email: "aisyah@gmail.com",  
    phone: "08123456789",  
    website: "aisyah.com",  
    company: {  
      name: "PT Akalpa Belt ",  
      catchPhrase: "Perempuan Berdaya",  
      bs: "transformasi Kekuatan Perempuan berdaya"  
    },  
    address: {  
      street: "Jl. Merdeka No. 123",  
      suite: "Apt 45",  
      city: "Jakarta",  
      zipcode: "12345",  
      geo: {  
        lat: "-6.2088",  
        lng: "106.8456"  
      }  
    }  
  };  
  
  try {  
    console.log("Mengirim data POST...", karyawanBaru);  
  
    loading.classList.remove('d-none');  
  
    const response = await fetch(API_URL, {  
      method: 'POST',  
      headers: {  
        'Content-Type': 'application/json',  
      },  
      body: JSON.stringify(karyawanBaru)  
    });  
  
    if (!response.ok) {  
      throw new Error(`HTTP error! status: ${response.status}`);  
    }  
  
    const dataHasil = await response.json();  
  
    console.log("Response dari server:", dataHasil);  
    console.log("Status Response:", response.status);  
  }  
}
```

```
        alert(`Berhasil menambahkan karyawan!\n\nID baru:
${dataHasil.id}\nNama: ${dataHasil.name}\nEmail:
${dataHasil.email}\n\nCatatan: JSONPlaceholder menyimpan data.`);

        const dataDenganIdBaru = { ...karyawanBaru, id: dataHasil.id };
        const tempCol = document.createElement('div');
        tempCol.className = 'col-md-4 mb-3';
        tempCol.innerHTML = `
            <div class="card h-100 shadow-sm border-success">
                <div class="card-body">
                    <h5 class="card-title"> ${dataDenganIdBaru.name} <span
class="badge bg-success">Baru</span></h5>
                    <p class="card-text text-muted">Email:
${dataDenganIdBaru.email}</p>
                    <p class="card-text">Perusahaan:
${dataDenganIdBaru.company.name}</p>
                    <p class="card-text"><small>Kota:
${dataDenganIdBaru.address.city}</small></p>
                    <button class="btn btn-sm btn-outline-primary"
onclick="cariKaryawan(${dataDenganIdBaru.id})">Detail Profil</button>
                </div>
            </div>
        `;
        container.prepend(tempCol);

    } catch (error) {
        console.error("Error saat POST data:", error);
        alert(`Gagal menambahkan data: ${error.message}`);
    } finally {
        loading.classList.add('d-none');
    }
}
```



Gambar F.1 – Menambahkan karyawan

F.2 Latihan 2 – Filter Array

Pertanyaan / Instruksi Latihan:

Latihan 2 (Filter Array): Sebelum data dirender ke layar dengan `forEach`, lakukan filter terlebih dahulu. Tampilkan HANYA karyawan yang tinggal di kota yang mengandung huruf "s" (misal: "Southside", "London"). Petunjuk: Gunakan `data.filter(...)` sebelum `data.forEach(...)`.

Jawaban / Kode Solusi:

```
const btnLoad = document.getElementById('btnLoad');
const container = document.getElementById('containerKaryawan');
const loading = document.getElementById('loading');

// Buat tombol Tambah
const btnTambah = document.createElement('button');
btnTambah.textContent = 'Tambah Karyawan Baru';
btnTambah.className = 'btn btn-success mb-3 me-2';
btnTambah.id = 'btnTambah';

// Masukkan ke dalam div yang sama dengan btnLoad
const parentDiv = btnLoad.parentElement;
parentDiv.appendChild(btnTambah);

// ===== KOLOM PENCARIAN KOTA =====
const searchDiv = document.createElement('div');
searchDiv.className = 'mb-3';
searchDiv.innerHTML = `
    <div class="row">
        <div class="col-md-6">
```

```

        <div class="input-group">
            <span class="input-group-text">Cari Kota</span>
            <input type="text" id="searchKota" class="form-control"
placeholder="Cari kota yang mengandung huruf...">
            <button id="btnCariKota" class="btn btn-outline-
primary">Cari</button>
            <button id="btnTampilkanSemua" class="btn btn-outline-
secondary">Tampilkan Semua</button>
        </div>
        <small class="text-muted">Contoh: Ketik "s" maka akan muncul
karyawan dari kota yang mengandung huruf "s"</small>
    </div>
</div>
`;
parentDiv.insertAdjacentElement('afterend', searchDiv);

// Ambil elemen search
const searchKota = document.getElementById('searchKota');
const btnCariKota = document.getElementById('btnCariKota');
const btnTampilkanSemua = document.getElementById('btnTampilkanSemua');

// Variabel untuk menyimpan data asli
let dataKaryawanAsli = [];

// Endpoint API
const API_URL = 'https://jsonplaceholder.typicode.com/users';

// ===== FUNGSI PENCARIAN KOTA DENGAN CONSOLE LOG =====
function cariBerdasarkanKota(data, keyword) {
    if (!keyword || keyword === '') {
        console.log("Menampilkan semua data (tanpa filter)");
        return data;
    }

    const keywordLower = keyword.toLowerCase();

    console.log("\n=====
=====");
    console.log("PENCARIAN: Mencari kota yang mengandung huruf:",
keyword);
    console.log("=====
=====");
    console.log("Total data yang diperiksa:", data.length, "karyawan\n");

    const hasilFilter = data.filter(function(karyawan) {
        const kota = karyawan.address?.city || '';

```

```

        const isMatch = kota.toLowerCase().includes(keywordLower);

        if (isMatch) {
            console.log("[COCOK]", karyawan.name, "- Kota:", kota);
        } else {
            console.log("[TIDAK COCOK]", karyawan.name, "- Kota:", kota);
        }

        return isMatch;
    });

    console.log("\n-----");
    console.log("HASIL PENCARIAN: Ditemukan", hasilFilter.length,
    "karyawan dari kota yang mengandung huruf", keyword);

    if (hasilFilter.length > 0) {
        console.log("\nDaftar kota yang cocok:");
        hasilFilter.forEach(function(karyawan, index) {
            console.log("  ", index + 1 + ".", karyawan.address?.city,
            "(", karyawan.name, ")");
        });
    } else {
        console.log("  Tidak ada kota yang mengandung huruf", keyword);
    }
    console.log("=====\n");

    return hasilFilter;
}

// ===== FUNGSI RENDER ULANG DENGAN FILTER =====
function renderDenganFilter() {
    const keyword = searchKota.value;
    const dataTerfilter = cariBerdasarkanKota(dataKaryawanAsli, keyword);

    if (dataTerfilter.length === 0 && keyword !== '') {
        container.innerHTML = `<div class="alert alert-info">Tidak ada
karyawan yang tinggal di kota mengandung huruf "${keyword}"</div>`;
        return;
    }

    renderData(dataTerfilter);
}

// Event listener untuk tombol Load

```

```

btnLoad.addEventListener('click', function() {
  loading.classList.remove('d-none');
  container.innerHTML = '';

  fetch(API_URL)
    .then(function(response) {
      if (!response.ok) {
        throw new Error('Gagal mengambil data');
      }
      return response.json();
    })
    .then(function(dataKaryawan) {
      console.log("\n=====
=====");
      console.log("DATA BERHASIL DIMUAT DARI API");
      console.log("=====
=====");
      console.log("Data asli:", dataKaryawan);
      console.log("=====
=====\\n");

      // Simpan data asli
      dataKaryawanAsli = dataKaryawan;

      // Kosongkan input search
      searchKota.value = '';

      // Tampilkan semua data
      renderData(dataKaryawanAsli);
    })
    .catch(function(error) {
      container.innerHTML = `<div class="alert alert-danger">Error:
${error.message}</div>`;
    })
    .finally(function() {
      loading.classList.add('d-none');
    });
});

// Event listener untuk tombol Cari Kota
btnCariKota.addEventListener('click', function() {
  if (dataKaryawanAsli.length === 0) {
    alert("Silakan muat data terlebih dahulu dengan klik 'Muat Data
dari API'");
    return;
  }
}

```

```
renderDenganFilter();
});

// Event listener untuk tombol Tampilkan Semua
btnTampilkanSemua.addEventListener('click', function() {
    searchKota.value = '';
    if (dataKaryawanAsli.length > 0) {
        console.log("\nMenampilkan semua data (filter direset)");
        renderData(dataKaryawanAsli);
    } else {
        container.innerHTML = '<div class="alert alert-info">Silakan muat
data terlebih dahulu</div>';
    }
});

// Bisa tekan Enter untuk mencari
searchKota.addEventListener('keypress', function(e) {
    if (e.key === 'Enter') {
        btnCariKota.click();
    }
});

function renderData(data) {
    if (!data || data.length === 0) {
        container.innerHTML = '<div class="alert alert-warning">Tidak ada
data karyawan</div>';
        return;
    }

    container.innerHTML = '';

    data.forEach(function(karyawan) {
        const col = document.createElement('div');
        col.className = 'col-md-4 mb-3';

        const companyName = karyawan.company?.name || 'Tidak tersedia';
        const cityName = karyawan.address?.city || 'Tidak tersedia';

        col.innerHTML = `
            <div class="card h-100 shadow-sm">
                <div class="card-body">
                    <h5 class="card-title">${karyawan.name}</h5>
                    <p class="card-text text-muted">Email:
${karyawan.email}</p>
                    <p class="card-text">Perusahaan: ${companyName}</p>
                </div>
            </div>
        `;
        container.appendChild(col);
    });
}
```



```

                <p class="card-text"><small>Kota:
${cityName}</small></p>
                <button class="btn btn-sm btn-outline-primary"
onclick="cariKaryawan(${karyawan.id})">Detail Profil</button>
            </div>
        </div>
    `;

    container.appendChild(col);
});
}

async function cariKaryawan(id) {
    try {
        console.log("\nMencari data ID:", id);

        const response = await
fetch(`https://jsonplaceholder.typicode.com/users/${id}`);

        if (!response.ok) {
            throw new Error('Data tidak ditemukan');
        }

        const data = await response.json();
        console.log("Ditemukan:", data);

        alert(`Ditemukan:\nNama: ${data.name}\nEmail:
${data.email}\nPerusahaan: ${data.company?.name || '-'}\nKota:
${data.address?.city || '-'} `);

        return data;

    } catch (error) {
        console.error("Error:", error.message);
        alert(`Error: ${error.message}`);
    }
}

// Event listener untuk tombol Tambah
btnTambah.addEventListener('click', function() {
    tambahKaryawanBaru();
});

async function tambahKaryawanBaru() {
    const karyawanBaru = {
        name: "Aisyah Ayu Salsabila",

```

```
username: "aisyah",
email: "aisyah@gmail.com",
phone: "08123456789",
website: "aisyah.com",
company: {
  name: "PT Akalpa Belt",
  catchPhrase: "Perempuan Berdaya",
  bs: "transformasi Kekuatan Perempuan berdaya"
},
address: {
  street: "Jl. Merdeka No. 123",
  suite: "Apt 45",
  city: "Semarang",
  zipcode: "12345",
  geo: {
    lat: "-6.2088",
    lng: "106.8456"
  }
}
};

try {
  console.log("\nMengirim data POST...", karyawanBaru);

  loading.classList.remove('d-none');

  const response = await fetch(API_URL, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
    },
    body: JSON.stringify(karyawanBaru)
  });

  if (!response.ok) {
    throw new Error(`HTTP error! status: ${response.status}`);
  }

  const dataHasil = await response.json();

  console.log("Response dari server:", dataHasil);
  console.log("Status Response:", response.status);

  alert(`Berhasil menambahkan karyawan!\n\nID baru:
  ${dataHasil.id}\nNama: ${dataHasil.name}\nEmail: ${dataHasil.email}`);
```

```

const dataDenganIdBaru = { ...karyawanBaru, id: dataHasil.id };

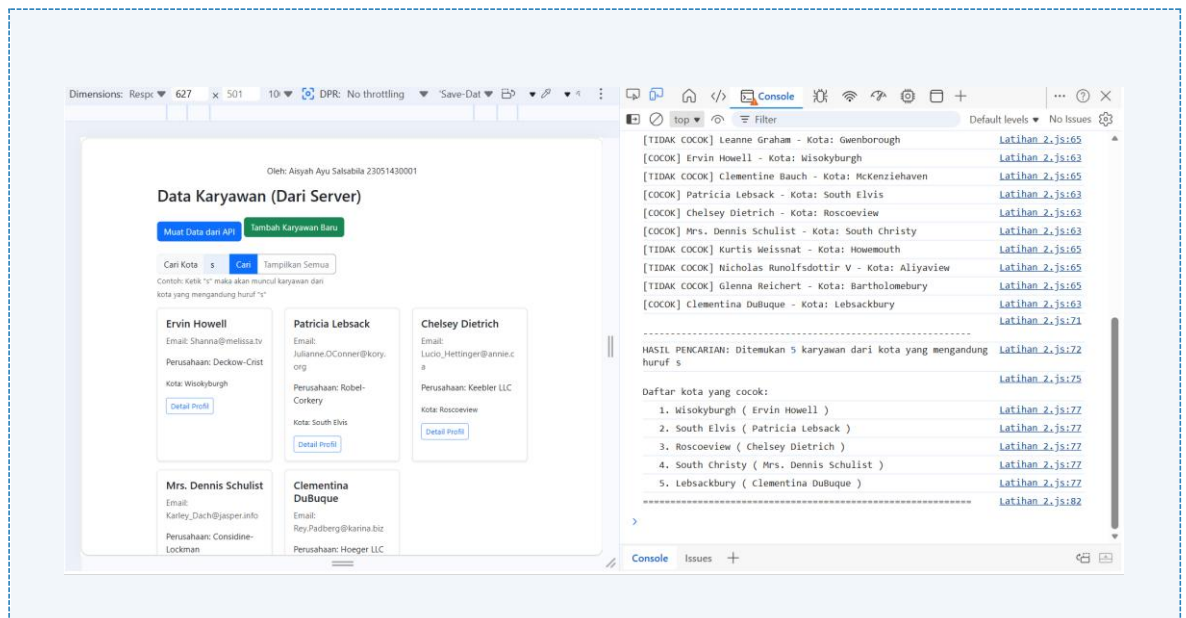
// Tambahkan ke data asli
dataKaryawanAsli.push(dataDenganIdBaru);

console.log("Data baru ditambahkan:", dataDenganIdBaru.name, "- Kota:", dataDenganIdBaru.address.city);

// Tampilkan data baru
renderData(dataKaryawanAsli);

} catch (error) {
  console.error("Error saat POST data:", error);
  alert(`Gagal menambahkan data: ${error.message}`);
} finally {
  loading.classList.add('d-none');
}
}

```



Gambar F.2 – Kota mengandung s Filter

F.3 Tugas Proyek Mini – (Integrasi Data Produksi)

Deskripsi proyek mini:

Tugas Proyek Mini (Integrasi Data Produksi): menganggap API `https://jsonplaceholder.typicode.com/posts` adalah database "Laporan Insiden". Ingin menampilkan daftar 10 laporan insiden terbaru (Title = Judul Insiden, Body = Deskripsi). Pada setiap card, tambahkan tombol "Tindak Lanjut". Saat diklik, `alert()` menampilkan "Tiket ID sedang diproses oleh Tim Maintenance".

Penjelasan pendekatan/solusi yang digunakan:

Menggunakan data dummy lokal berbahasa Indonesia agar praktikum lebih relevan dan tidak tergantung koneksi internet, Mengambil 10 data terbaru dengan method slice(-10) dan mengurutkan dari ID tertinggi ke terendah, Menampilkan data dalam bentuk card Bootstrap dengan judul insiden, deskripsi, dan tombol "Tindak Lanjut" di setiap card., Tombol "Tindak Lanjut" menampilkan alert berisi pesan "Tiket ID sedang diproses oleh Tim Maintenance" sesuai permintaan soal.

Jawaban untuk JS:

```
// Data dummy (15 insiden)
const dataInsiden = [
  {id:1, userId:101, title:"Kebocoran Pipa Air Lantai 3", body:"Air
menggenangi koridor lantai 2 dan 3. Perbaiki segera."},
  {id:2, userId:102, title:"Listrik Padam di Ruang Server",
body:"Server mati total akibat korsleting. Tim IT sedang perbaiki."},
  {id:3, userId:103, title:"Kerusakan Lift Barang", body:"Lift barang
macet. Teknisi menunggu suku cadang."},
  {id:4, userId:104, title:"Kebakaran Kecil di Dapur Kantin",
body:"Kompas gas meledak. Api sudah dipadamkan."},
  {id:5, userId:105, title:"Genangan Air di Parkiran", body:"Pompa
air mati, 5 kendaraan terendam."},
  {id:6, userId:101, title:"AC Ruang Rapat Tidak Berfungsi", body:"AC
mati total, ruangan panas 32°C."},
  {id:7, userId:106, title:"Pintu Otomatis Rusak", body:"Pintu masuk
timur tidak bisa terbuka."},
  {id:8, userId:107, title:"Kebocoran Gas di Laboratorium", body:"Bau
gas menyengat. Area dievakuasi."},
  {id:9, userId:102, title:"Jaringan Internet Putus", body:"Internet
gedung A dan B putus total."},
  {id:10, userId:108, title:"Kerusakan Mesin Fotokopi", body:"Mesin
error kode E-47."},
  {id:11, userId:103, title:"Pipa Bocor di Ruang Arsip", body:"Air
menetes dari plafon. Dokumen basah."},
  {id:12, userId:109, title:"Lampu Parkiran Mati", body:"5 tiang
lampu parkir mati."},
  {id:13, userId:104, title:"CCTV Mati", body:"20% kamera tidak
berfungsi."},
  {id:14, userId:105, title:"Toilet Pria Tersumbat", body:"Toilet
lantai 2 mampet total."},
  {id:15, userId:106, title:"Genset Darurat Rusak", body:"Genset
tidak mau menyala."}
];

const container = document.getElementById('containerInsiden');
```

```

const loading = document.getElementById('loadingSpinner');
const btnMuat = document.getElementById('btnMuatInsiden');

btnMuat.onclick = async () => {
    loading.style.display = 'inline-block';
    container.innerHTML = '<div class="col-12 text-center py-5"><div
class="spinner-border text-danger"></div><p>Memuat data...</p></div>';

    await new Promise(r => setTimeout(r, 500));

    const terbaru = dataInsiden.slice(-10).sort((a,b) => b.id - a.id);
    container.innerHTML = '';

    terbaru.forEach(i => {
        const desc = i.body.length > 100 ? i.body.slice(0,100)+'...' :
i.body;
        container.innerHTML += `
            <div class="col-md-4 mb-3">
                <div class="card h-100">
                    <div class="card-body">
                        <span class="badge bg-danger mb-2">Insiden
#${i.id}</span>
                        <h5 class="text-
danger">${escapeHtml(i.title)}</h5>
                        <p class="small">${escapeHtml(desc)}</p>
                        <button class="btn btn-warning w-100"
onclick="alert('Tiket ID ${i.id} sedang diproses oleh Tim
Maintenance.\\nJudul: ${escapeHtml(i.title)}')">Tindak Lanjut</button>
                    </div>
                </div>
            </div>
        `;
    });

    loading.style.display = 'none';
};

function escapeHtml(str) {
    return str.replace(/[\&<>]/g, function(m) {
        if(m === '&') return '&amp;';
        if(m === '<') return '&lt;';
        if(m === '>') return '&gt;';
        return m;
    });
}

```

Kode HTML lengkap:

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Laporan Insiden</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0-alpha1/dist/css/bootstrap.min.css" rel="stylesheet">
  <style>
    body { background: #f4f6f9; }
    .card { border-left: 5px solid #dc3545; transition: 0.2s; }
    .card:hover { transform: translateY(-3px); }
  </style>
</head>
<body class="container mt-4">

  <div class="d-flex justify-content-between align-items-center mb-4">
    <h1 class="text-danger">Laporan Insiden</h1>
    <div id="loading" class="spinner-border text-danger" style="display:none"></div>
  </div>

  <div id="container" class="row">
    <div class="col-12 text-center">
      <button id="btnLoad" class="btn btn-danger btn-lg">Muat 10
Insiden Terbaru</button>
    </div>
  </div>

  <footer class="mt-5 text-center text-muted small">Tim Maintenance -
Tindak lanjut dalam 1x24 jam</footer>

  <script>
    // Data dummy singkat (10 insiden saja)
    const dataInsiden = [
      {id:1, userId:101, title:"Kebocoran Pipa Air Lantai 3",
body:"Air menggenangi koridor lantai 2 dan 3. Perbaikan segera
diperlukan."},
      {id:2, userId:102, title:"Listrik Padam di Ruang Server",
body:"Server mati total akibat korsleting. Tim IT sedang memperbaiki."},
      {id:3, userId:103, title:"Kerusakan Lift Barang",
body:"Lift barang macet. Teknisi menunggu suku cadang dari pabrik."},
```

```

        {id:4, userId:104, title:"Kebakaran Kecil di Dapur Kantin",
body:"Kompore gas meledak. Api sudah dipadamkan, dapur rusak."},
        {id:5, userId:105, title:"Genangan Air di Parkiran Bawah",
body:"Pompa air mati, 5 kendaraan terendam. Air sedang dikuras."},
        {id:6, userId:101, title:"AC Ruang Rapat Tidak Berfungsi",
body:"AC mati total, ruangan panas 32°C. Rapat terpaksa dipindah."},
        {id:7, userId:106, title:"Pintu Otomatis Rusak",
body:"Pintu masuk timur tidak bisa terbuka. Teknisi datang besok."},
        {id:8, userId:107, title:"Kebocoran Gas di Laboratorium",
body:"Bau gas menyengat. Area dievakuasi, tim damkar turun."},
        {id:9, userId:102, title:"Jaringan Internet Putus",
body:"Internet gedung A dan B putus total. 200 karyawan terdampak."},
        {id:10, userId:108, title:"Kerusakan Mesin Fotokopi",
body:"Mesin error kode E-47. Perlu penggantian drum unit baru."},
        {id:11, userId:103, title:"Pipa Bocor di Ruang Arsip",
body:"Air menetes dari plafon. Dokumen penting basah terkena air."},
        {id:12, userId:109, title:"Lampu Parkiran Mati", body:"5
tiang lampu parkiran mati. Kondisi gelap dan berbahaya."},
        {id:13, userId:104, title:"CCTV Mati", body:"20% kamera
tidak berfungsi. Rekaman seminggu terakhir hilang."},
        {id:14, userId:105, title:"Toilet Pria Tersumbat",
body:"Toilet lantai 2 mampet total. Air kotor meluap ke koridor."},
        {id:15, userId:106, title:"Genset Darurat Rusak",
body:"Genset tidak mau menyala. Baterai soak dan starter motor rusak."}
    ];

    const container = document.getElementById('container');
    const loading = document.getElementById('loading');
    const btnLoad = document.getElementById('btnLoad');

    btnLoad.onclick = async () => {
        loading.style.display = 'inline-block';
        container.innerHTML = '<div class="col-12 text-center py-5"><div class="spinner-border text-danger"></div><p>Memuat data...</p></div>';

        await new Promise(r => setTimeout(r, 500)); // simulasi delay

        const terbaru = dataInsiden.slice(-10).sort((a,b) => b.id - a.id);
        container.innerHTML = '';

        terbaru.forEach(i => {
            const desc = i.body.length > 100 ?
i.body.slice(0,100)+'...' : i.body;

```

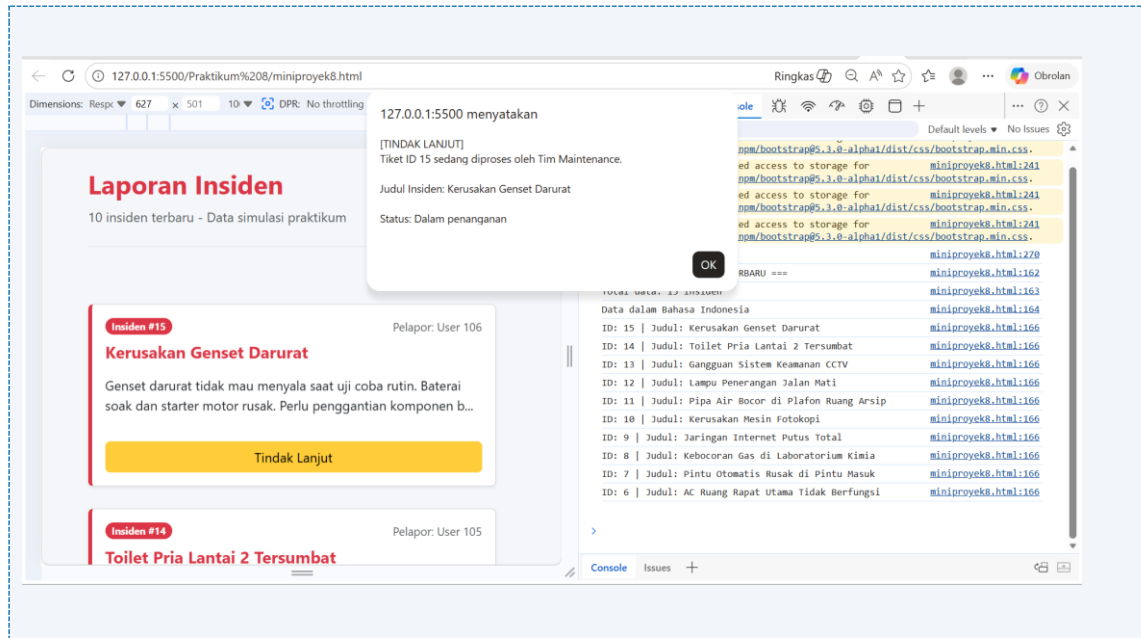
```

        container.innerHTML += `
            <div class="col-md-4 mb-3">
                <div class="card h-100">
                    <div class="card-body">
                        <span class="badge bg-danger mb-
2">Insiden #${i.id}</span>
                        <h5 class="text-
danger">${escapeHtml(i.title)}</h5>
                        <p
class="small">${escapeHtml(desc)}</p>
                        <button class="btn btn-warning w-100"
onclick="alert('Tiket ID ${i.id} sedang diproses oleh Tim
Maintenance.\nJudul: ${escapeHtml(i.title)}')">Tindak Lanjut</button>
                    </div>
                </div>
            </div>
        `;
    });

    loading.style.display = 'none';
};

function escapeHtml(str) {
    return str.replace(/&lt;&gt;/g, function(m) {
        if (m === '&') return '&amp;';
        if (m === '<') return '&lt;';
        if (m === '>') return '&gt;';
        return m;
    });
}
</script>
</body>
</html>

```

Gambar F.3 – checklis 5 s

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Sesuai, seluruh fitur yang diimplementasikan telah berjalan sesuai dengan teori yang dipelajari. Data dummy berhasil ditampilkan dalam bentuk card dengan urutan 10 insiden terbaru, tombol "Tindak Lanjut" berfungsi menampilkan alert sesuai permintaan, serta aspek teknis seperti manipulasi array, DOM, event handling, dan keamanan data telah diterapkan dengan tepat. Semua fitur berjalan seperti yang diharapkan.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Perbaikan syntax error karena block fetch if tidak didalam kurung	<pre>if (!response.ok) { throw new Error('Gagal mengambil data'); }</pre>
Input jumlah kosong atau huruf tidak terdeteksi	Gunakan <code>parseInt()</code> + <code>isNaN()</code> untuk validasi
Tombol dengan ID sama (dobel)	Hanya gunakan satu ID unik per elemen
Menghitung jumlah checkbox yang dicentang	Gunakan <code>filter(item => item === true).length</code>
Menampilkan checklist yang tercentang	Simpan dalam array lalu <code>.join(", ")</code>

G.3 Kaitan dengan Penerapan di Industri

Sistem monitoring insiden dapat diterapkan di industri manufaktur untuk mencatat dan menindaklanjuti kerusakan mesin secara real-time. Pemanfaatan array method mendukung pengolahan data log yang efisien, sementara fitur "Tindak Lanjut" dengan alert mensimulasikan ticketing otomatis yang mempercepat respons tim maintenance. Penerapan loading state dan dynamic rendering mencerminkan prinsip cyber-physical system (CPS), di mana antarmuka digital terhubung langsung dengan data operasional mesin untuk pengambilan keputusan cepat dan tepat.

H. KESIMPULAN

1	Asynchronous JavaScript dengan <code>async/await</code> dan <code>fetch()</code> terbukti esensial untuk mengambil data dari server tanpa menghambat proses lain, menggunakan konsep Promise.
2	Manipulasi Array menggunakan method <code>slice()</code> untuk memfilter data dan <code>sort()</code> untuk mengurutkan merupakan implementasi nyata dari pengolahan data secara immutable.
3	Manipulasi DOM dengan <code>innerHTML</code> dan <code>forEach()</code> menunjukkan bagaimana data dapat dirender secara dinamis ke halaman web tanpa perlu reload.
4	Event Handling melalui <code>addEventListener</code> dan properti <code>onclick</code> menjadi dasar interaktivitas pengguna dalam merespon aksi klik pada elemen HTML.
5	Keamanan Data dengan fungsi <code>escapeHtml()</code> membuktikan pentingnya sanitasi input untuk mencegah serangan Cross-Site Scripting (XSS) pada aplikasi web.

I. DAFTAR PUSTAKA

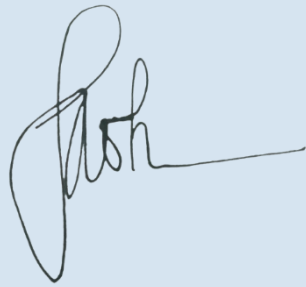
No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	Chambers, J. M. (2014). <i>Object-Oriented Programming , Functional Programming and R</i> . 29(2), 167–180. https://doi.org/10.1214/13-STS452
[3]	Gunawan, R., Suherman, Y., & Wibowo, S. S. (2022). <i>Rancang Bangun Sistem Informasi Verifikasi Dan Validasi Data Pengajuan Tender Berbasis Web Menggunakan Model Terstruktur dan UML Menurut Instruksi Presiden Nomor 3 Tahun mengembangkan</i> . 16(62), 10–19.
[4]	Mulawarman Munsyir, S.E., S.Si., M. K., Eko Prasetyo Widhi, M. K., Eko Wahyudi, M. T., Dr. Bagus Setya Rintyarna, M. K., & Vandika, A. Y. (2024). <i>Belajar Algoritma dan Pemrograman dengan Javascript</i> .

[5]	Sastradipraja, C. K., & Munawar, Z. (2022). Konsep dasar teknologi web. Kaizen Media Publishing. https://books.google.co.id/books?id=E16YEAAAQBAJ
[6]	Gunawan, R., Suherman, Y., & Wibowo, S. S. (2022). <i>Rancang Bangun Sistem Informasi Verifikasi Dan Validasi Data Pengajuan Tender Berbasis Web Menggunakan Model Terstruktur dan UML Menurut Instruksi Presiden Nomor 3 Tahun mengembangkan</i> . 16(62), 10–19.

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Diperiksa oleh Asisten / Dosen



Aisyah Ayu Salsabila
NIM: 23051430001

Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT.
Tanggal: _____

PANDUAN DAN KETENTUAN PENGISIAN LAPORAN

Baca dengan seksama sebelum mengisi template laporan ini

1. Aturan Umum

a.	Laporan dikerjakan secara INDIVIDUAL, kecuali ada instruksi khusus dari dosen.
b.	Isi laporan harus merupakan hasil pekerjaan sendiri. Plagiarisme (menyalin laporan teman secara identik) akan mendapatkan nilai NOL (0) untuk kedua pihak.
c.	Bagian Dasar Teori HARUS ditulis dengan kalimat dan bahasa sendiri, bukan disalin mentah dari modul atau internet.
d.	Screenshot WAJIB memperlihatkan nama/NIM mahasiswa pada salah satu bagian tampilan (bisa di judul halaman web atau bagian lain) sebagai bukti keaslian pekerjaan.
e.	Semua teks dalam kotak yang berwarna abu-abu/biru muda dan bersifat miring adalah instruksi yang HARUS diganti dengan isi yang sesungguhnya.

2. Format File dan Ketentuan Pengumpulan

Hal	Ketentuan
Format File	Microsoft Word (.docx). Konversi ke PDF jika diminta dosen.
Nama File	Laporan_Modul[No]_[NIM]_[NamaLengkap].docx Contoh: Laporan_Modul02_12320001_AhmadDhani.docx
Ukuran Kertas	A4 (210 × 297 mm)
Margin	Atas: 3 cm Bawah: 3 cm Kiri: 4 cm Kanan: 3 cm
Font Teks Isi	Arial, ukuran 11pt, spasi 1.15
Font Kode Program	Courier New, ukuran 10pt, latar gelap (sudah tersedia di template)
Batas Pengumpulan	7 hari kalender setelah tanggal pelaksanaan praktikum
Media Pengumpulan	Github repository mahasiswa + konfirmasi email ke dosen
Subjek Email	[WebLab-No.Modul] NIM - Nama Lengkap Contoh: [WebLab-02] 12320001 - Ahmad Dhani

3. Kompilasi Laporan Akhir Semester

PENTING – LAPORAN AKHIR SEMESTER

Pada akhir semester, seluruh laporan modul (Modul 1–12, kecuali Modul 13 UTS dan Modul 16 UAS) akan dikompilasi menjadi SATU BERKAS LAPORAN AKHIR. Ketentuan kompilasi:

a.	Laporan setiap modul dicetak/disimpan sebagai satu dokumen terpisah sesuai template ini.
b.	Kompilasi disatukan dengan urutan: Halaman Cover Akhir → Daftar Isi → Laporan Modul 1 → Laporan Modul 2 → ... → Laporan Modul 12 → Laporan Modul 14 & 15.
c.	Laporan UTS (Modul 13) dan UAS (Modul 16) dikumpulkan secara TERPISAH dengan ketentuan yang diumumkan saat ujian.
d.	Setiap laporan modul di dalam kompilasi diawali dengan halaman Cover (Bagian A di template ini) sebagai pemisah antar modul.
e.	Nama file laporan akhir: LaporanAkhir_[NIM]_[NamaLengkap].docx
f.	Laporan akhir dikumpulkan paling lambat H-3 sebelum UAS.

4. Ketentuan Pengumpulan via GitHub

```

PraktikumWebMob_TekInd_2026/
├── README.md
├── Laporan/
│   ├── Laporan_Modul01_[NIM].docx
│   ├── Laporan_Modul02_[NIM].docx
│   └── ... (dst hingga Modul 15)
├── Pertemuan01_[NIM]_[Nama]/
│   ├── index.html
│   └── style.css
├── Pertemuan02_[NIM]_[Nama]/
└── ... (dst)

```

Pastikan setiap Commit memiliki pesan yang deskriptif. Contoh: "Menambahkan Laporan Modul 02 – CSS Dasar". Lakukan Push ke GitHub setiap akhir praktikum.

5. Konsekuensi Keterlambatan Pengumpulan

Keterlambatan	Konsekuensi
1–2 hari	Pengurangan nilai 10% dari total skor laporan
3–4 hari	Pengurangan nilai 25% dari total skor laporan
5–7 hari	Pengurangan nilai 50% dari total skor laporan
> 7 hari	Laporan TIDAK DITERIMA (nilai 0) – wajib temui dosen untuk klarifikasi
Tidak dikumpulkan	Nilai laporan = 0, mahasiswa tidak diperkenankan mengikuti UAS

— Selamat mengerjakan —